

Selenium WebDriver

Comprehensive Interview Preparation

Senior SDET / Test Automation Lead — Worldline Round 2

WHAT THIS PDF CONTAINS

100 interview questions, senior-level answers, Java coding examples, framework architecture, waits, exceptions, Grid, parallel execution, POM/PageFactory, Selenium 4, debugging, CI/CD, flaky tests and leadership scenarios.

Study style: Understand → explain in your own words → code → discuss trade-offs.

Contents

- 1. Fundamentals Q1–10
- 2. Locators & WebElements Q11–18
- 3. Waits Q19–28
- 4. Exceptions & Debugging Q29–36
- 5. Windows/Frames/Alerts Q37–44
- 6. Actions/JS/Cookies/Screenshots Q45–52
- 7. Selenium 4 Q53–60
- 8. POM/PageFactory Q61–67
- 9. Framework Architecture Q68–76
- 10. Parallel/Grid Q77–84
- 11. Flaky Tests Q85–90
- 12. Senior SDET/Lead Q91–100
- 13. Coding Reference
- 14. Final Revision

1. Selenium Fundamentals

Q1. What is Selenium?

Interview-ready answer: An open-source browser-automation ecosystem. WebDriver provides APIs to control real browsers. It is mainly for web UI automation.

Q2. What are Selenium's main components?

Interview-ready answer: WebDriver, Selenium Grid and the Selenium IDE ecosystem. For senior roles, WebDriver and Grid are the most important.

Q3. What is WebDriver?

Interview-ready answer: The API/protocol-based interface used by automation code to control a browser.

Q4. Explain Selenium WebDriver architecture.

Interview-ready answer: Test code → Selenium WebDriver API → browser driver/WebDriver implementation → browser → web application.

Q5. Why is modern WebDriver preferred over Selenium RC?

Interview-ready answer: It uses modern browser automation standards and provides a cleaner, more reliable architecture than the legacy RC approach.

Q6. What is the W3C WebDriver standard?

Interview-ready answer: A standard browser-automation protocol defining communication between automation clients and browser implementations.

Q7. What is Selenium Manager?

Interview-ready answer: Selenium's built-in driver/browser management capability that can reduce manual driver setup in modern Selenium.

Q8. Selenium vs Playwright?

Interview-ready answer: Selenium has a mature WebDriver ecosystem and Grid; Playwright provides modern browser control, auto-waiting and browser-context isolation. Choose based on requirements.

Q9. Can Selenium automate native mobile apps?

Interview-ready answer: No. Selenium targets web browsers; Appium is commonly used for native/hybrid mobile applications.

Q10. What are Selenium limitations?

Interview-ready answer: Web-focused, requires good synchronisation/locator/data design, and does not itself provide a complete test runner or reporting platform.

2. Locators & WebElements

Q11. What locator strategies are available?

Interview-ready answer: id, name, class name, tag name, link text, partial link text, CSS and XPath. Prefer stable test attributes where available.

Q12. CSS vs XPath?

Interview-ready answer: CSS is concise and readable; XPath is more expressive for relationships/text. Choose the simplest stable locator.

Q13. Why avoid absolute XPath?

Interview-ready answer: It depends on DOM hierarchy and breaks easily when markup changes.

Q14. What makes a good locator?

Interview-ready answer: Stable, unique, readable and based on meaningful attributes; data-testid is often excellent.

Q15. findElement vs findElements?

Interview-ready answer: findElement returns one element and throws if absent; findElements returns a list and returns an empty list when nothing matches.

Q16. How do you handle dynamic elements?

Interview-ready answer: Use stable attributes/relative conditions and explicit waits for the required state.

Q17. isDisplayed vs isEnabled vs isSelected?

Interview-ready answer: They validate visibility, enabled state and selection state respectively.

Q18. What is WebElement?

Interview-ready answer: A Selenium representation of a DOM element with actions and state methods.

3. Waits & Synchronisation

Q19. Why do we need waits?

Interview-ready answer: Modern UIs are asynchronous; elements can appear, become enabled or be replaced later. Wait for meaningful conditions.

Q20. Implicit vs explicit wait?

Interview-ready answer: Implicit is global for element lookup; explicit targets a specific condition. Prefer a deliberate explicit-wait strategy.

Q21. Why avoid Thread.sleep()?

Interview-ready answer: Fixed delays waste time when the app is ready early and can still be too short when it is slow.

Q22. What is FluentWait?

Interview-ready answer: A configurable wait with timeout, polling interval and ignored exceptions.

Q23. What is ExpectedConditions?

Interview-ready answer: Reusable conditions for visibility, presence, clickability, alerts, frames, titles and more.

Q24. Presence vs visibility?

Interview-ready answer: Presence means in DOM; visibility means present and visible.

Q25. What does elementToBeClickable mean?

Interview-ready answer: Visible and enabled; it does not guarantee that an overlay cannot intercept the click.

Q26. How do you build a reusable wait utility?

Interview-ready answer: Centralise WebDriverWait methods so page objects do not duplicate synchronisation logic.

Q27. Should implicit and explicit waits be mixed?

Interview-ready answer: Prefer not to; mixing can make timing harder to reason about.

Q28. How do you handle AJAX-driven UI changes?

Interview-ready answer: Wait for a state change such as element visibility, spinner disappearance, text/state change or URL change.

4. Selenium Exceptions & Debugging

Q29. What is NoSuchElementException?

Interview-ready answer: The locator did not find an element in the current page/context. Check locator, timing, iframe, page state and DOM.

Q30. What is StaleElementReferenceException?

Interview-ready answer: The DOM changed and the stored element reference is no longer valid. Re-locate and synchronise.

Q31. How do you handle ElementClickInterceptedException?

Interview-ready answer: Find the blocking overlay/state, wait for it to disappear and then interact normally.

Q32. What causes ElementNotInteractableException?

Interview-ready answer: The element exists but is hidden, disabled or otherwise not actionable.

Q33. How do you debug a flaky Selenium test?

Interview-ready answer: Capture screenshot/source/logs, check timing, locators, data, environment, parallelism and application behaviour, then classify root cause.

Q34. Should we catch all Selenium exceptions?

Interview-ready answer: No. Catch only when meaningful recovery/context exists; otherwise fail clearly and preserve evidence.

Q35. How do you handle TimeoutException?

Interview-ready answer: Treat it as evidence that the condition was not reached. Investigate condition, locator, app, environment and timing before increasing timeout.

Q36. How do you avoid hiding failures in utilities?

Interview-ready answer: Do not swallow exceptions or return false success. Log useful context and rethrow/fail when recovery is impossible.

5. Windows, Frames, Alerts & Browser Controls

Q37. How do you handle multiple windows/tabs?

Interview-ready answer: Capture handles, wait for the new handle, switch deliberately, then return to the required handle.

Q38. How do you switch to an iframe?

Interview-ready answer: `driver.switchTo().frame(...)`, interact, then `defaultContent()` or `parentFrame()`.

Q39. How do you handle nested iframes?

Interview-ready answer: Switch parent frame first, then child frame; unwind the context deliberately.

Q40. How do you handle JavaScript alerts?

Interview-ready answer: Wait for alert presence, switch to Alert, read text and accept/dismiss/sendKeys as required.

Q41. How do you handle navigation?

Interview-ready answer: Use `get/navigate` and synchronise with the resulting application state.

Q42. How do you handle authentication pop-ups?

Interview-ready answer: Use an application/browser-supported authentication strategy; avoid insecure credentials embedded in URLs.

Q43. How do you handle dropdowns?

Interview-ready answer: Use `Select` for native HTML select elements; interact with custom dropdown DOM for JavaScript widgets.

Q44. How do you upload a file?

Interview-ready answer: For input[type=file], send the absolute path to the file input.

6. Actions, JavaScript, Cookies, Screenshots & Shadow DOM

Q45. What is Actions used for?

Interview-ready answer: Advanced mouse/keyboard interactions such as hover, drag-and-drop, click-and-hold and key combinations.

Q46. When would you use JavaScriptExecutor?

Interview-ready answer: For browser-side tasks such as scrolling or inspection when normal WebDriver APIs are insufficient. Prefer normal WebDriver clicks first.

Q47. How do you scroll to an element?

Interview-ready answer: Use Selenium interaction/scrolling or JS scrollIntoView where appropriate.

Q48. How do you work with cookies?

Interview-ready answer: getCookies, addCookie, getCookieNamed, deleteCookie and deleteAllCookies.

Q49. How do you take screenshots?

Interview-ready answer: Use TakesScreenshot and integrate automatic capture with the test listener/hook.

Q50. How do you capture screenshots on failure?

Interview-ready answer: Use TestNG/JUnit listener or equivalent hook and attach evidence to the report.

Q51. How do you capture page source/browser evidence?

Interview-ready answer: getPageSource plus browser logging/CDP/BiDi mechanisms where supported.

Q52. What is Shadow DOM?

Interview-ready answer: Encapsulated DOM content. Open shadow roots can be accessed through Selenium's ShadowRoot/SearchContext APIs.

7. Selenium 4 & Modern WebDriver

Q53. Important Selenium 4 features?

Interview-ready answer: W3C alignment, relative locators, Grid 4, new tab/window APIs, improved DevTools integration and Selenium Manager.

Q54. What are relative locators?

Interview-ready answer: Spatial relationships such as above, below, toLeftOf, toRightOf and near.

Q55. How do you open a new tab/window in Selenium 4?

Interview-ready answer: Use driver.switchTo().newWindow(WindowType.TAB/WINDOW).

Q56. What is Selenium Grid 4?

Interview-ready answer: A distributed Selenium infrastructure for running WebDriver sessions across machines/browsers.

Q57. Why use Grid?

Interview-ready answer: Parallel execution, cross-browser coverage and scalable execution.

Q58. What is CDP?

Interview-ready answer: Chrome DevTools Protocol; it enables browser-specific capabilities such as network or browser inspection.

Q59. What is WebDriver BiDi?

Interview-ready answer: A bidirectional browser automation protocol designed to provide richer event-driven capabilities alongside WebDriver.

Q60. Grid vs cloud browser providers?

Interview-ready answer: Grid gives infrastructure control; cloud providers offer managed scalable browser infrastructure. Decide based on security, cost, coverage and operations.

8. POM & PageFactory

Q61. What is Page Object Model?

Interview-ready answer: A design pattern that encapsulates page locators and interactions so tests remain business-readable and maintainable.

Q62. What makes a good Page Object?

Interview-ready answer: Stable locators, business-level actions, minimal duplication and clear separation of UI interaction from test intent.

Q63. POM vs PageFactory?

Interview-ready answer: POM is a design pattern; PageFactory is a Selenium support mechanism for @FindBy element fields.

Q64. Disadvantages of PageFactory?

Interview-ready answer: Proxy-based lifecycle can be less explicit, especially with dynamic DOMs. By locators often give clearer control.

Q65. How do you handle stale elements with PageFactory?

Interview-ready answer: Avoid long-lived element references; re-locate and synchronise with the current DOM.

Q66. Should Page Objects contain assertions?

Interview-ready answer: Keep most assertions in validation/test layers; page state helpers such as isLoggedIn can be useful.

Q67. How do you avoid huge Page Objects?

Interview-ready answer: Create reusable component objects for headers, tables, menus and widgets.

9. Framework Architecture & Reusability

Q68. Explain scalable Selenium framework architecture.

Interview-ready answer: Separate tests, pages/components, driver lifecycle, config, data, utilities, listeners/reporting and CI concerns.

Q69. Where should WebDriver be created?

Interview-ready answer: In a central driver factory/lifecycle layer; inject/pass it to page objects.

Q70. How do you manage environments?

Interview-ready answer: Externalise URLs, browser, timeouts and other configuration; do not hardcode environment values.

Q71. How do you manage secrets?

Interview-ready answer: Use Jenkins/cloud secret stores/environment variables; never hardcode credentials in source.

Q72. How do you design reusable utilities?

Interview-ready answer: One responsibility per utility; avoid a giant catch-all Utils class.

Q73. How do you integrate with Jenkins?

Interview-ready answer: Run through Maven/Gradle, pass environment parameters, publish reports and archive diagnostics.

Q74. How do you integrate reporting?

Interview-ready answer: Use Allure/Extent or equivalent listeners/hooks and attach screenshots/logs/environment details.

Q75. How do you reduce framework flakiness?

Interview-ready answer: Stable locators, explicit waits, isolated data, deterministic setup, clean driver lifecycle and controlled parallelism.

Q76. How do you review automation code?

Interview-ready answer: Review locators, waits, assertions, exceptions, independence, duplication, security and maintainability.

10. Parallel Execution & Selenium Grid

Q77. How do you run Selenium tests in parallel safely?

Interview-ready answer: Independent WebDriver sessions, isolated data and thread-safe lifecycle.

Q78. Why ThreadLocal?

Interview-ready answer: It gives each thread its own driver reference and prevents parallel tests from controlling the same session.

Q79. How do you clean up ThreadLocal?

Interview-ready answer: Quit the driver and call ThreadLocal.remove() in teardown.

Q80. What causes parallel failures?

Interview-ready answer: Shared drivers/data, static state, order dependencies, infrastructure limits and application race conditions.

Q81. How do you decide what can run in parallel?

Interview-ready answer: Run independent tests in parallel; redesign or serialise flows with shared state.

Q82. How do you size Grid parallelism?

Interview-ready answer: Measure CPU, memory, network, application capacity and queue time; increase concurrency based on evidence.

Q83. What is RemoteWebDriver?

Interview-ready answer: A WebDriver client that creates/controls a browser session on a remote Selenium server/Grid.

Q84. How do you troubleshoot Grid failures?

Interview-ready answer: Check nodes, capacity, browser/driver compatibility, network, Grid logs and infrastructure health.

11. Flaky Tests & Debugging Strategy

Q85. What is a flaky test?

Interview-ready answer: A test whose result is inconsistent without an intentional application/test change.

Q86. Why does a test fail only in CI?

Interview-ready answer: Compare browser/configuration, timing, data, parallelism, network/environment and application behaviour.

Q87. What if a test passes locally but fails headless?

Interview-ready answer: Compare headless config, viewport, rendering, timing, permissions and environment; do not blindly add sleeps.

Q88. What if it fails after 20 successful runs?

Interview-ready answer: Capture evidence and investigate data, environment, recent changes, resource leaks and timing.

Q89. Should flaky tests be retried?

Interview-ready answer: Only in a controlled, observable way. Indiscriminate retry hides defects.

Q90. How do you measure automation quality?

Interview-ready answer: Stable pass rate, flake rate, execution time, meaningful coverage, escaped defects and diagnosis/maintenance effort.

12. Senior SDET / Lead Scenarios

Q91. How would you improve an existing Selenium framework?

Interview-ready answer: Baseline flakiness, runtime, coverage and maintenance; then improve locators, waits, data, lifecycle, components, reporting and parallelism.

Q92. How would you migrate Selenium to Playwright?

Interview-ready answer: Define measurable goals, pilot, migrate incrementally, compare runtime/flakiness and retire old code after confidence.

Q93. Developer says the test is wrong. What do you do?

Interview-ready answer: Stay evidence-driven: reproduce, compare requirements and actual behaviour, share diagnostics and resolve ambiguity with product.

Q94. Release has failing automation. What do you do?

Interview-ready answer: Classify failures by product/automation/environment/data and communicate business risk with evidence.

Q95. How do you decide what to automate?

Interview-ready answer: Prioritise stable, repeatable, high-value regression and critical business flows.

Q96. How do you improve coverage?

Interview-ready answer: Use risk-based positive/negative/boundary/role/integration scenarios, not raw test-count targets.

Q97. How do you mentor junior engineers?

Interview-ready answer: Teach reasoning and framework principles through pairing, code reviews and progressively increasing ownership.

Q98. How do you handle technical disagreement?

Interview-ready answer: Define criteria, compare evidence, run a small proof of concept if needed and document the decision.

Q99. How would you reduce a four-hour regression suite?

Interview-ready answer: Profile first; remove duplication, improve data/setup, parallelise independent tests and optimise expensive operations.

Q100. What makes you a Senior SDET rather than only an automation coder?

Interview-ready answer: Quality engineering: risk-based testing, maintainable automation, debugging, CI feedback, mentoring and engineering decisions based on evidence.

13. Coding Reference — Core Selenium Patterns

Driver Factory

```
public final class DriverFactory {
    private DriverFactory() {}

    public static WebDriver createDriver() {
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--start-maximized");
        return new ChromeDriver(options);
    }
}
```

Explicit Wait Utility

```
public class WaitUtils {
    private final WebDriverWait wait;

    public WaitUtils(WebDriver driver) {
        wait = new WebDriverWait(driver, Duration.ofSeconds(15));
    }

    public WebElement clickable(By locator) {
        return wait.until(
            ExpectedConditions.elementToBeClickable(locator));
    }
}
```

ThreadLocal WebDriver

```
private static final ThreadLocal<WebDriver> DRIVER =
    new ThreadLocal<>();

public static WebDriver getDriver() {
    return DRIVER.get();
}

public static void removeDriver() {
    WebDriver driver = DRIVER.get();
    if (driver != null) driver.quit();
    DRIVER.remove();
}
```

POM Example

```
public class LoginPage {
    private final WebDriver driver;
    private final By user = By.id("username");
    private final By password = By.id("password");
    private final By login = By.id("login");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void login(String u, String p) {
        driver.findElement(user).sendKeys(u);
        driver.findElement(password).sendKeys(p);
        driver.findElement(login).click();
    }
}
```

Window Handling

```
String parent = driver.getWindowHandle();
for (String handle : driver.getWindowHandles()) {
    if (!handle.equals(parent)) {
        driver.switchTo().window(handle);
        break;
    }
}
```

Screenshot

```
byte[] image = ((TakesScreenshot) driver)
    .getScreenshotAs(OutputType.BYTES);
```

14. Last-Minute Selenium Revision

- **Locator:** stable, unique, readable; prefer test attributes.
- **Wait:** condition-based, not fixed sleep.
- **Stale:** DOM changed; re-locate and synchronise.
- **Click intercepted:** investigate overlays/state before JavaScript.
- **Frames:** switch context deliberately.
- **Windows:** use handles and wait for new context.
- **POM:** business-readable page actions and encapsulated locators.
- **PageFactory:** optional mechanism, not the POM pattern.
- **Parallel:** separate driver + isolated data.
- **ThreadLocal:** isolates driver references, not all application state.
- **Grid:** distribute sessions and measure infrastructure capacity.
- **Flaky:** investigate root cause; don't hide with retries.
- **CI:** screenshots/logs/browser/environment evidence.
- **Senior answer:** explain why, trade-offs, failure modes and maintainability.

WORLDLINE INTERVIEW MINDSET

For senior questions, do not stop at 'how'. Explain why you choose the approach, what can fail, how you diagnose it, how it behaves in parallel/CI, and how you keep the framework maintainable.